

Real-Time E-Commerce Analytics Platform

Goal

Build a production-style Data Engineering project that simulates an e-commerce company processing real-time business events through Kafka and Spark Streaming, storing operational data in PostgreSQL, analytical data in BigQuery, and exposing business insights through a React dashboard.

Objectives

This project demonstrates:

- Data Engineering fundamentals
- Real-time stream processing
- ETL and ELT pipelines
- Data warehousing
- Cloud deployment
- Dashboard development
- System design
- Dockerized infrastructure

Target Resume Keywords:

- Apache Kafka
 - Apache Spark
 - Spark Streaming
 - ETL Pipelines
 - Data Warehousing
 - BigQuery
 - PostgreSQL
 - FastAPI
 - Docker
 - React
 - Data Analytics
 - Cloud Computing
-

High Level Architecture

Data Flow:

Event Generator → Kafka → Spark Streaming → PostgreSQL

Nightly ETL: PostgreSQL → Airflow → BigQuery

Analytics Layer: BigQuery → FastAPI → React Dashboard

Business Scenario

The platform simulates an online shopping company.

Events generated:

- Product Views
- Orders
- Payments
- Inventory Updates
- Returns

Management wants:

- Revenue Tracking
 - Product Analytics
 - Customer Analytics
 - Inventory Monitoring
 - Real-Time Monitoring
-

Technology Stack

Data Engineering

- Python
- Apache Kafka
- Apache Spark
- Spark Structured Streaming
- Apache Airflow

Databases

- PostgreSQL
- BigQuery

Backend

- FastAPI

Frontend

- React
- Chart.js

Cloud

- Google Cloud Platform

DevOps

- Docker
- Docker Compose
- GitHub Actions

Infrastructure

Containers:

1. zookeeper
2. kafka
3. spark-master
4. spark-worker
5. postgres
6. airflow
7. fastapi
8. react

Everything starts using:

`docker compose up`

No local Kafka installation.

No local Spark installation.

Everything containerized.

Phase 1 - Event Generation

Goal

Simulate realistic e-commerce traffic.

Events

Order Event

```
{ "order_id": "ORD001", "customer_id": "C001", "product_id": "P001", "quantity": 2, "price": 1499,
  "timestamp": "2026-06-10T10:00:00" }
```

Payment Event

```
{ "payment_id": "PAY001", "order_id": "ORD001", "amount": 2998, "method": "UPI" }
```

Product View Event

```
{ "customer_id": "C001", "product_id": "P001" }
```

Return Event

```
{ "order_id": "ORD001", "reason": "Damaged Product" }
```

Inventory Event

```
{ "product_id": "P001", "stock_change": -2 }
```

Event Generator Requirements

Generate:

- 20 orders/sec
- 50 views/sec
- 5 payments/sec
- 1 return/sec

Libraries:

- Faker
- kafka-python

Phase 2 - Kafka Layer

Topics

orders

payments

product_views

returns

inventory_updates

Responsibilities

- Event ingestion
- Event durability
- Event buffering
- Decoupling producer and consumer

Deliverables

- Kafka Producer
 - Kafka Topics
 - Kafka Monitoring
-

Phase 3 - Spark Streaming

Goal

Process events in real time.

Read Source

Kafka Topics

Validation Rules

- price > 0
- quantity > 0
- valid customer id

- valid product id

Cleaning

- Remove null records
- Remove malformed records

Enrichment

Add:

- day
- month
- year
- hour
- weekday

Aggregations

Revenue Per Hour

Orders Per Hour

Revenue Per Product

Revenue Per Category

Return Rate

Average Order Value

Customer Spend

Top Products

Output

Write processed data into PostgreSQL.

PostgreSQL Schema

customers

customer_id UUID PK

name

email

city

signup_date

products

product_id UUID PK

product_name

category

price

stock_quantity

orders

order_id UUID PK

customer_id FK

order_timestamp

order_status

total_amount

order_items

order_item_id UUID PK

order_id FK

product_id FK

quantity

item_price

payments

payment_id UUID PK

order_id FK

payment_method

amount

payment_status

returns

return_id UUID PK

order_id FK

return_reason

return_date

product_views

view_id UUID PK

customer_id FK

product_id FK

viewed_at

Why PostgreSQL?

Purpose:

Operational Database (OLTP)

Stores:

- Current Orders
- Customers
- Products
- Payments

Used by:

- FastAPI
 - Real-Time Dashboard
 - Operational Queries
-

Phase 4 - Airflow

Goal

Move operational data into warehouse.

Schedule:

Every night

Pipeline:

Extract → Transform → Load

Tasks:

- Extract orders
- Extract customers
- Extract products
- Data cleaning
- Warehouse loading

Phase 5 - BigQuery Warehouse

Goal

Business Intelligence and Analytics

Fact Table

fact_orders

Columns:

order_key

customer_key

product_key

date_key

quantity

revenue

profit

discount

Dimension Tables

dim_customer

customer_key

city

signup_date

dim_product

product_key

category

product_name

dim_date

date_key

month

quarter

year

dim_region

region_key

city

state

country

Why BigQuery?

Purpose:

Analytical Database (OLAP)

Used For:

- Reporting
- Dashboards
- Analytics
- KPIs

Examples:

Revenue by Month

Revenue by Category

Customer Segmentation

Top Products

Not used for application transactions.

Phase 6 - FastAPI

Endpoints

/api/kpis

/api/revenue

/api/products

/api/customers

/api/orders

/api/realtime

/api/inventory

Phase 7 - React Dashboard

Dashboard Page

Cards:

- Revenue Today
- Orders Today
- Active Customers
- Return Rate

Charts:

- Revenue Trend
 - Orders Trend
-

Sales Analytics

Charts:

- Daily Revenue
- Weekly Revenue
- Monthly Revenue

Filters:

- Date
 - Category
 - Region
-

Product Analytics

Charts:

- Top Products
 - Revenue by Category
 - Product Performance
-

Customer Analytics

Charts:

- Customer Segmentation
 - Top Cities
 - Customer Spend
-

Inventory Analytics

Metrics:

- Stock Levels
 - Low Inventory
 - Out of Stock
-

Real-Time Monitoring

Live Metrics:

- Kafka Throughput
- Events/sec
- Orders/sec
- Spark Processing Rate

Pipeline Status:

- Healthy
 - Warning
 - Failed
-

Data Quality Dashboard

Metrics:

- Failed Records
 - Duplicate Records
 - Missing Values
-

Deployment

Frontend

Vercel

Backend

Cloud Run

Database

Neon PostgreSQL

Warehouse

BigQuery

Object Storage

Google Cloud Storage

Stretch Goals

Anomaly Detection

Detect:

- Revenue Drop
 - Revenue Spike
 - Return Spike
-

Email Alerts

Trigger:

Inventory below threshold

Scheduled Reports

Generate:

- Daily PDF
 - Weekly PDF
 - Monthly PDF
-

Authentication

Google OAuth

Role-Based Access

Roles:

- Admin
- Analyst

Timeline

Week 1

Docker Setup

Kafka

PostgreSQL

Event Generator

Week 2

Spark Streaming

Kafka Consumers

Data Processing

Week 3

FastAPI

React Dashboard

Week 4

Airflow

BigQuery

Warehouse Design

Week 5

Cloud Deployment

Monitoring

Testing

Week 6

Advanced Features

Documentation

Architecture Diagrams

Resume Update

Final Resume Bullet

Built a cloud-native real-time e-commerce analytics platform using Apache Kafka, Spark Structured Streaming, PostgreSQL, Airflow, and BigQuery to process streaming business events, implement ETL pipelines, and deliver real-time operational and analytical dashboards through FastAPI and React.